DevLedger



(Personal Developer Utility & Asset Vault)

Portfolio of Evidence – Final PoE

AI Usage Write-Up

Module: Programming 3D / Open-Source Coding (Intermediate)

Module Code: PROG7314 / OPSC7312

Team Members:
Blake Derek Godfrey | ST10435415
Syed Muhammad Hamza Kazmi | ST10443021
Connor Albertyn | ST10437293
Muzammil Cassim | ST10259792

Institution: The Independent Institute of Education

Academic Year: 2026

Word Count: 483

AI Use Write-Up: HustleHub+ DevLedger (PROG7314 Final PoE)

Overview

My team, Quantum Coders, developed DevLedger ourselves, including the Kotlin code, REST API, database architecture, artwork and documentation. We used generative AI (Claude, by Anthropic) in three ways: double-checking requirements and deliverables, working through deployment issues, and getting design ideas for our dashboard graph.

How we used it

1. Verifying requirements and deliverables

On a big project something can always get missed, particularly something more visible in the rubric than the task statement. We gave Claude the full PoE brief and rubric and used it as a second set of eyes in three ways:

- **Requirement cross-checking:** We asked it to turn the specifications into a checklist covering biometric authentication, offline mode with synchronisation, Firebase Cloud Messaging notifications, multi-language support for South African languages, production assets and our user-defined features. We then tested each item against our application.
- **Deliverables checking:** We verified that everything on the submission list was present: a README with the video link, release notes, signed production APK, Play Store screenshots, GitHub Actions workflow, system logging, commented code, Final POE tag and a repository with no zip files.
- **Rubric alignment:** We benchmarked our work against the top mark band for each criterion (commit history, collaboration evidence and demonstration video points) and closed the gaps this revealed.

2. Deployment queries

We hit a problem deploying our REST API to Render because our repository sits under our organisation's GitHub account. Claude described the process: authorise the Render app for the organisation, give it access to the repository, and ensure the connecting user has permission. We did this ourselves in the Render and GitHub settings, deployed, and checked it against Render's documentation, since platform settings change.

3. Dashboard graph redesign

Our dashboard graph was hard to read and inconsistent with the rest of our UI. We described the issue to Claude and asked for design improvements. It recommended a less cluttered layout, better labelling and more uniform colour use. We treated these as inspiration only: we coded the new graph ourselves, adapted it to our data and theme, discarded suggestions that didn't apply, and checked the result on a real Android device.

What AI didn't do

Claude did not write our code, build our features, create our assets or record our demonstration. When it spotted a potential gap or proposed an idea, we verified it by testing on a device, checking the repository or reading the documentation, and decided whether it belonged in the release. We did not blindly trust its output.

Reflection

Using AI as a checker and sounding board helped us submit a more complete product and resolve a deployment blocker faster. It did not replace our own testing or understanding. We can explain and defend every part of the app, as the voice-over in our demonstration video shows.

Reference

Anthropic (2026) *Claude* [Large language model]. Available at: **https://claude.ai** (Accessed: 5 November 2026).